Detecting DoS Attack using Suricata in Kali Linux

Objective

The objective of this lab is to practically demonstrate how the Suricata Intrusion Detection System (IDS) can detect and log a Denial-of-Service (DoS) attack. This lab will teach students how to configure Suricata, understand network traffic, simulate a DoS attack using hping3, analyze logs, and understand IDS rule-based detection. Upon completion of this lab, you will be able to: • Understand DoS attacks and their characteristics.

- Configure and run Suricata IDS.
- Generate different kinds of attack traffic.
- Analyze Suricata alert logs.
- Create custom rules for detecting anomalous traffic.

Introduction

Denial-of-Service (DoS) attacks remain one of the most disruptive threats in modern network security. These attacks aim to overwhelm a target system or service with excessive traffic, ultimately causing slowdowns, service interruptions, or complete system unavailability. With the increasing dependency on networked applications and online services, early detection of DoS attacks has become a critical requirement for maintaining system reliability and security.

Suricata—an open-source, high-performance Intrusion Detection and Prevention System (IDPS)—plays a vital role in identifying such attack patterns. Equipped with deep packet inspection, multi-threading capabilities, and rule-based detection, Suricata can monitor network traffic in real time and generate alerts upon detecting suspicious or abnormal behaviour. Its ability to analyze packet signatures and anomalies makes it a powerful tool against DoS, DDoS, and other network-based attacks.

In this project, Kali Linux is used as the attack-generating platform, while Suricata is deployed on the victim system to detect and log malicious traffic. By simulating a DoS attack using tools such as hping3**,** this project demonstrates how Suricata identifies attack signatures, logs alerts in files like fast.log and eve.json**,** and provides useful insights for network administrators.

The purpose of this project is to understand the behaviour of DoS attacks, configure Suricata for intrusion detection, analyze logs generated during the attack, and showcase how real-time monitoring can be used to mitigate network threats. This hands-on environment reinforces key concepts of cybersecurity, intrusion detection, and network traffic analysis.

What is Suricata?

Suricata is a free, open-source IDS/IPS cybersecurity tool that acts as both an Intrusion Detection System (IDS) and an Intrusion Prevention System (IPS). It is used by organizations all around the world to detect cyber threats and monitor networks for suspicious activity.

Suricata's strength lies in its versatility. When tuned correctly, it is a high-performance tool that can handle large volumes of network traffic and generate vast amounts of network traffic data. It is also extremely flexible, offering deep analysis of various protocols and the ability to customize rule sets to fit your organization's specific needs. Because it's an open-source IDS/IPS, Suricata benefits from a large, active community that constantly develops and refines its capabilities.

Suricata works by analyzing network traffic and checking it against a database of existing rules or signatures. When traffic matches a signature, an alert is issued signaling the user to initiate a response

,

action. Suricata rules lists can be custom-made or imported from third-party sources.

Put simply, Suricata is a powerful and adaptable tool that provides a robust layer of defense for any organization's network security strategy.

How does Suricata work?

Simply put, Suricata works by monitoring traffic and issuing alerts whenever that traffic matches the signature of a known threat. Here is a more detailed breakdown:

- **Network Traffic:** Acquisition: Suricata operates in a sniffing or promiscuous mode on a designated network interface. This allows it to capture all network traffic flowing through that interface, regardless of its intended recipient.
- **Packet Parsing and Analysis:** Suricata employs packet capture libraries to collect network traffic in the form of raw data packets. It then utilizes packet parsing libraries to dissect these packets into headers, payloads, and protocol-specific data structures.
- **Signature Matching:** Suricata is programmed with a rule set or signature database. These signatures define patterns that match specific network activity associated with known threats. Suricata employs pattern-matching techniques to compare the extracted data from the packets against the signatures in the rule set.
- **Deep Packet Inspection (Optional):** Suricata can be configured for deep packet inspection (DPI). During DPI, Suricata examines the payload portion of the packet beyond the headers. This enables the detection of threats that might hide malicious content within the data while also providing a robust set of network security monitoring (NSM) data.
- **Action and Logging:** Upon detecting a match or anomaly, Suricata triggers pre-defined actions based on its configuration. These actions can include logging the event for further investigation, generating alerts for security personnel, or even blocking the traffic flow if Suricata is deployed in Intrusion Prevention System (IPS) mode.

The effectiveness of Suricata depends on two main factors:

- **Maintained Rule Sets:** The signature database requires regular updates to incorporate the latest threats. Fortunately, Suricata benefits from a community that actively contributes to maintaining and expanding the available signatures.
- **Configuration and Customization:** Suricata offers a high degree of configurability. Security teams can tailor its operation to focus on specific network traffic types or ports. They can even develop custom rules to address unique threats specific to their network environment.

What is a rule in Suricata?

A rule in Suricata is essentially an instruction that defines what kind of network traffic to look for and what action to take if it's found. It's like a blueprint for Suricata to identify potential threats. Here's what the general Suricata rule format looks like:

- **Action:** This specifies what Suricata should do when it encounters traffic matching the rule's criteria. Common actions include logging the event, generating alerts, or even blocking the traffic (if Suricata is in Intrusion Prevention System mode).
- **Header:** This section defines the characteristics of the network traffic Suricata should focus on. It can specify elements like:
- **Protocol:** (e.g., TCP, UDP, ICMP)
- **Source and Destination IP addresses/subnets**
- **Source and Destination Ports**
- **Direction of traffic (incoming, outgoing, or both)**
- **Options (Optional):** This section provides additional filters or conditions for Suricata to consider when evaluating traffic. It can include things like:
- **Payload content:** Matching specific strings or patterns within the data portion of the packet.

,

- **TCP flags:** Analyses specific flags set in the TCP header for certain behaviors.

How do you write rules in Suricata?
Writing your own Suricata rules can be tricky. For beginners, we recommend reading "The Security Analyst's Guide to Suricata" by Stamus Networks to get a better understanding of the process of writing custom Suricata rules.

When practicing, you could also use a Suricata rule generator. Some Suricata rule generators have been developed and released on GitHub, but we recommend using the Suricata Language Server.

The Suricata Language Server™ (SLS) adds rule (also known as signature) syntax checking, rule-writing hints, auto-completion, and performance guidance to your preferred editor. An open-source project developed and supported by Stamus Networks, SLS helps Suricata users write better, more effective, and more advanced rules.

What protocols are used in Suricata?

Suricata can handle a wide range of protocols to effectively monitor and analyze network traffic for suspicious activity, including but not limited to:

1. **Basic Protocols:**

    - TCP (Transmission Control Protocol)
    - UDP (User Datagram Protocol)
    - ICMP (Internet Control Message Protocol)
    - IP (Internet Protocol)

2. **Application Layer Protocols (Layer 7):**

    - HTTP (Hypertext Transfer Protocol)
    - HTTP/2:
    - FTP (File Transfer Protocol):
    - TLS/SSL (Transport Layer Security/Secure Sockets Layer):
    - SMB (Server Message Block):
    - DNS (Domain Name System):

3. **Other Supported Protocols:**

    - Dcerpc (Distributed Computing Environment Remote Procedure Call):
    - DHCP (Dynamic Host Configuration Protocol):
    - SSH (Secure Shell):
    - Many more

What is an IDS?

Intrusion Detection System (IDS) observes network traffic for malicious transactions and sends immediate alerts when it is observed. It is software that checks a network or system for malicious activities or policy violations. Each illegal activity or violation is often recorded either centrally using an SIEM system or notified to an administration. IDS monitors a network or system for malicious activity and protects a computer network from unauthorized access from users, including perhaps insiders. The

intrusion detector learning task is to build a predictive model (i.e. a classifier) capable of distinguishing

,

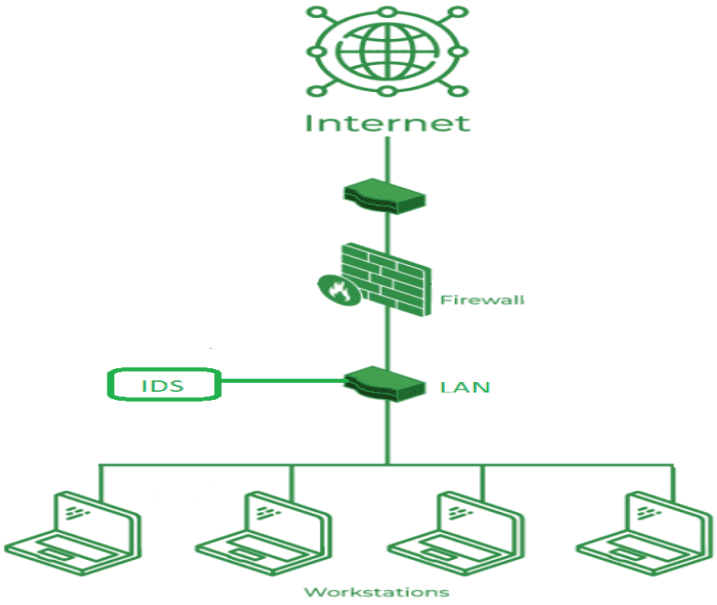
between ‘bad connections’ (intrusion/attacks) and ‘good (normal) connections’.

Common Methods of Intrusion

- **Address Spoofing:** Hiding the source of an attack by using fake or unsecured proxy servers making it hard to identify the attacker.
- **Fragmentation:** Sending data in small pieces to slip past detection systems.
- **Pattern Evasion:** Changing attack methods to avoid detection by IDS systems that look for specific patterns.
- **Coordinated Attack:** Using multiple attackers or ports to scan a network, confusing the IDS and making it hard to see what is happening.

Working of Intrusion Detection System (IDS)

- An IDS (Intrusion Detection System) monitors the traffic on a computer network to detect any suspicious activity.
- It analyzes the data flowing through the network to look for patterns and signs of abnormal behavior.
- The IDS compares the network activity to a set of predefined rules and patterns to identify any activity that might indicate an attack or intrusion.
- If the IDS detects something that matches one of these rules or patterns, it sends an alert to the system administrator.
- The system administrator can then investigate the alert and take action to prevent any damage or further intrusion.



Intrusion Detection System (IDS)

Classification of Intrusion Detection System (IDS)

Intrusion Detection System are classified into 5 types:

- **Network Intrusion Detection System (NIDS):** Network intrusion detection systems (NIDS) are set up at a planned point within the network to examine traffic from all devices on the network. It performs an observation of passing traffic on the entire subnet and matches the traffic that is passed on the subnets to the collection of known attacks. Once an attack is identified or abnormal behavior is observed, the alert can be sent to the administrator. An example of a NIDS is installing it on the subnet where firewalls

,

are located in order to see if someone is trying to crack the firewall.

- **Host Intrusion Detection System (HIDS):** Host intrusion detection systems (HIDS) run on independent hosts or devices on the network. A HIDS monitors the incoming and outgoing packets from the device only and will alert the administrator if suspicious or malicious activity is detected. It takes a snapshot of existing system files and compares it with the previous snapshot. If the analytical system files were edited or deleted, an alert is sent to the administrator to investigate. An example of HIDS usage can be seen on mission-critical machines, which are not expected to change their layout.
- **Hybrid Intrusion Detection System:** Hybrid intrusion detection system is made by the combination of two or more approaches to the intrusion detection system. In the hybrid intrusion detection system, the host agent or system data is combined with network information to develop a complete view of the network system. The hybrid intrusion detection system is more effective in comparison to the other intrusion detection system. Prelude is an example of Hybrid IDS.
- **Application Protocol-Based Intrusion Detection System (APIDS):** An application Protocol-based Intrusion Detection System (APIDS) is a system or agent that generally resides within a group of servers. It identifies the intrusions by monitoring and interpreting the communication on application-specific protocols. For example, this would monitor the SQL protocol explicitly to the middleware as it transacts with the database in the web server.
- **Protocol-Based Intrusion Detection System (PIDS):** It comprises a system or agent that would consistently reside at the front end of a server, controlling and interpreting the protocol between a user/device and the server. It is trying to secure the web server by regularly monitoring the HTTPS protocol stream and accepting the related HTTP protocol. As HTTPS is unencrypted and before instantly entering its web presentation layer then this system would need to reside in this interface, between to use the HTTPS.
- **Signature-Based Detection:** Signature-based detection checks network packets for known patterns linked to specific threats. A signature-based IDS compares packets to a database of attack signatures and raises an alert if a match is found. Regular updates are needed to detect new threats, but unknown attacks without signatures can bypass this system

Intrusion Detection System Evasion Techniques

- **Fragmentation:** Dividing the packet into smaller packet called fragment and the process is known as fragmentation. This makes it impossible to identify an intrusion because there can't be a malware signature.
- **Packet Encoding:** Encoding packets using methods like Base64 or hexadecimal can hide malicious content from signature-based IDS.
- **Traffic Obfuscation:** By making message more complicated to interpret, obfuscation can be utilised to hide an attack and avoid detection.
- **Encryption:** Several security features such as data integrity, confidentiality, and data privacy, are provided by encryption. Unfortunately, security features are used by malware developers to hide attacks and avoid detection.

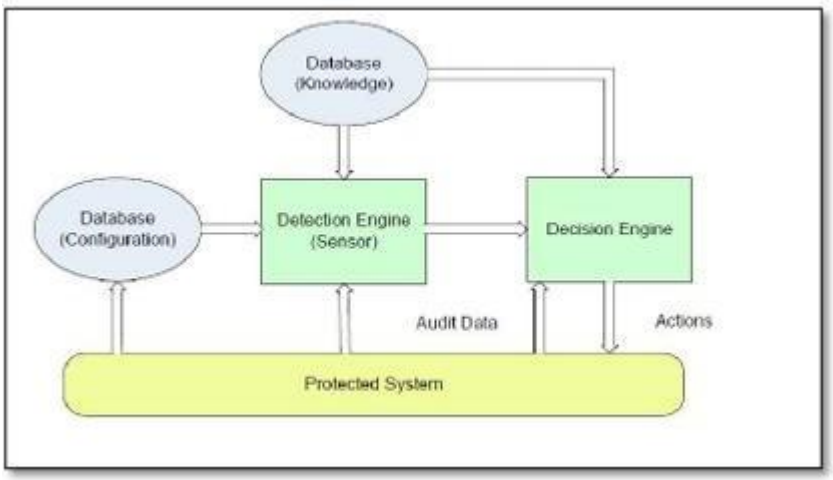
Detection Method of IDS

- **Signature-Based Method:** Signature-based IDS detects the attacks on the basis of the specific patterns such as the number of bytes or a number of 1s or the number of 0s in the network traffic. It also detects on the basis of the already known malicious instruction sequence that is used by the malware. The detected patterns in the IDS are known as signatures. Signature-based IDS can easily detect the attacks whose pattern (signature) already exists in the system but it is quite difficult to detect new malware attacks as their pattern (signature) is not known.
- **Anomaly-Based Method:** Anomaly-based IDS was introduced to detect unknown malware attacks as new malware is developed rapidly. In anomaly-based IDS there is the use of machine learning to create a trustful activity model and anything coming is compared with that model and it is declared suspicious if it is not found in the model. The machine learning-based method has a better-generalized property in comparison to signature-based IDS as these models can be trained according to the applications and

,

hardware configurations.

Components & Working of IDS



- **Database Knowledge Block** - Contains rules, signatures, and algorithms needed to detect threats and vulnerabilities. It is constantly updated with new threat intelligence.
- **Database Configuration Block** - Contains information about how the IDS communicates with the database, such as the type, location, and login credentials to ensure that the IDS can access the database and get important threat information.
- **Detection Engine** - Monitors network traffic or system activity (Audit) and generates alerts when it detects potential threats.
- **Decision Engine** - Receives alerts from the Detection Engine and decides how to respond (Decision) to them based on a set of policies and rules.
- **Protected System** - It is the component of the IDS that defends against attacks and monitors for potential security breaches. It may be a system, network, server or IT infrastructure. The Protected System may also be set to communicate with the Decision Engine in order to provide feedback on the effectiveness of the IDS and helps to develop its detection and response capabilities.

Importance & Need of IDS

- The primary importance of an intrusion detection system is that it alerts when an attack or intrusion is detected.
- As the use of networks and the internet increases, Cyber-attacks are also on the rise. This makes it crucial for organizations to have effective intrusion detection and prevention methods to safeguard their networks and data against unauthorized access and attacks.
- An intrusion detection system (IDS) analyzes network traffic and system activities for suspicious or unauthorized activities that may indicate an attack or suspected intrusion.
- So, organizations can monitor network traffic and system activity in real time, detect potential security breaches, and take appropriate measures to prevent them from causing harm by deploying an IDS.

Why Intrusion Detection Systems are Important ?

Cyberattacks are always increasing in complexity and sophistication, and Zero Day Attacks are common. As a result, network protection technologies must keep pace with new threats, and businesses must maintain high levels of security.

The objective is to assure secure, trusted communication of information. Therefore, an IDS is important to the security ecosystem. It operates as a defense for systems security when other technologies fail.

Identify security incidents.

,

• Analyze the quantity and types of attacks.
• Help identify bugs or problems with device configurations.
• Support regulatory compliance (by means of better network visibility and IDS log documentation).
• Improve security responses (by means of inspecting data within network packets, rather than manual census of systems).
• While IDSes are useful, they are extended in impact when coupled with IPSes. Intrusion Prevention Systems (IPS) add the ability to block threats.

Types of IDS

Intrusion Detection Systems are broadly classified into two types, each with its specific focus and functionality:

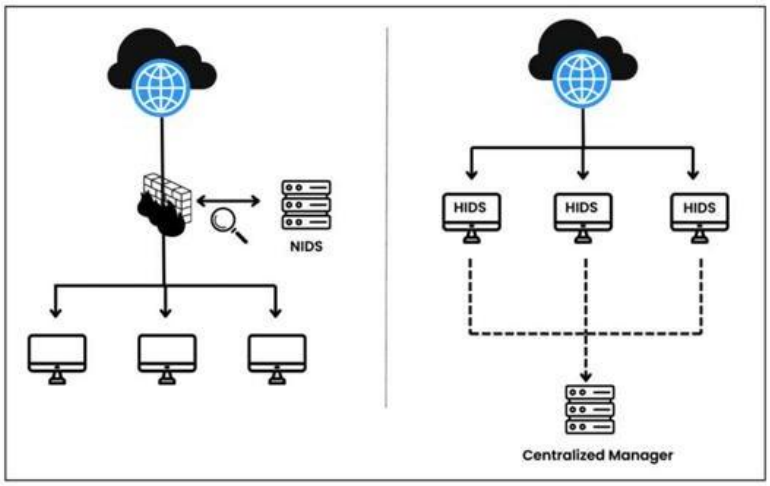
1. **Network-based IDS (NIDS):**

- Monitors network traffic for anomalies or intrusion attempts.
- Detects threats such as port scans, denial-of-service (DoS) attacks, and malicious network activities.
- Operates at central points within the network infrastructure, analyzing traffic from multiple devices.

2. **Host-based IDS (HIDS):**

- Monitors activities on individual devices or endpoints.
- Detects unauthorized access, file system changes, and suspicious processes.
- Operates locally on the host machine, providing granular visibility into system-specific events.
-

Both NIDS and HIDS play complementary roles in a robust security architecture, ensuring comprehensive monitoring across the network and individual devices. There are many more different IDS, like Protocol based Intrusion Detection System (PIDS), Application Protocol based Intrusion Detection System (APIDS), Hybrid Intrusion Detection System.



Intrusion Prevention System (IPS)

An Intrusion Prevention System is also known as an Intrusion Detection and Prevention System. It is a network security application that monitors network or system activities for malicious activity. The major functions of intrusion prevention systems are to identify malicious activity, collect information about it, report it, and attempt to block or stop it.

Intrusion Prevention System

Intrusion prevention systems are contemplated as an augmentation of Intrusion Detection Systems (IDS) because both IPS and IDS monitor network traffic and system activities for malicious activity.

An IPS typically records information related to observed events, notifies security administrators of important events, and produces reports. Many IPS can also respond to a detected threat by attempting to prevent it from succeeding. They use various response techniques, such as stopping the attack itself, changing the security environment, or altering the attack's content.

,

How Does an IPS Work?

An IPS works by analyzing network traffic in real-time and comparing it against known attack patterns and signatures. When the system detects suspicious traffic, it blocks it from entering the network.

1. Inline Deployment

An IPS is placed directly in the path of network traffic between your internal network and the internet or right behind a firewall. This position allows it to inspect every packet that flows through and immediately block anything suspicious before it causes harm.

2. Traffic Preprocessing

Before deep inspection begins, the IPS organizes the raw data.

It normalizes traffic formats to prevent attackers from hiding threats using clever encoding tricks.

It reassembles fragmented packets so that no data is missed.

3. Layered Packet Inspection

The IPS doesn't just look at surface-level information. This "deep packet inspection" helps the IPS understand both the technical structure and the intent behind the traffic. It dives into:

Network layer: Where packets come from and where they're going.

Transport layer: How reliable or suspicious the connection looks.

Application layer: What kind of data is being transmitted — like login info or file transfers.

4. Detection Mechanisms

Once packets are unpacked and understood, the IPS checks them using different techniques:

Signature-Based Detection: Matches traffic against known attack patterns (like a fingerprint database).

Anomaly-Based Detection: Flags anything that deviates from normal behavior — like an unusual spike in traffic.

Behavior-Based Detection: Watches for odd or suspicious actions over time, such as repeated failed logins.

Policy-Based Detection: Uses rules set by administrators — for example, blocking specific file types or countries.

5. Automated Response Actions

If the IPS detects a threat, it can act instantly

- Drop malicious packets
- Block the source IP
- Terminate sessions
- Trigger alerts or logs
- Update firewall rules

6. Tuning and Maintenance

An effective IPS isn't a "set it and forget it" system. It needs regular updates and fine-tuning to:

Keep up with evolving threats

Reduce false positives (blocking safe traffic by mistake)

Optimize performance under heavy loads

Objective of the Project

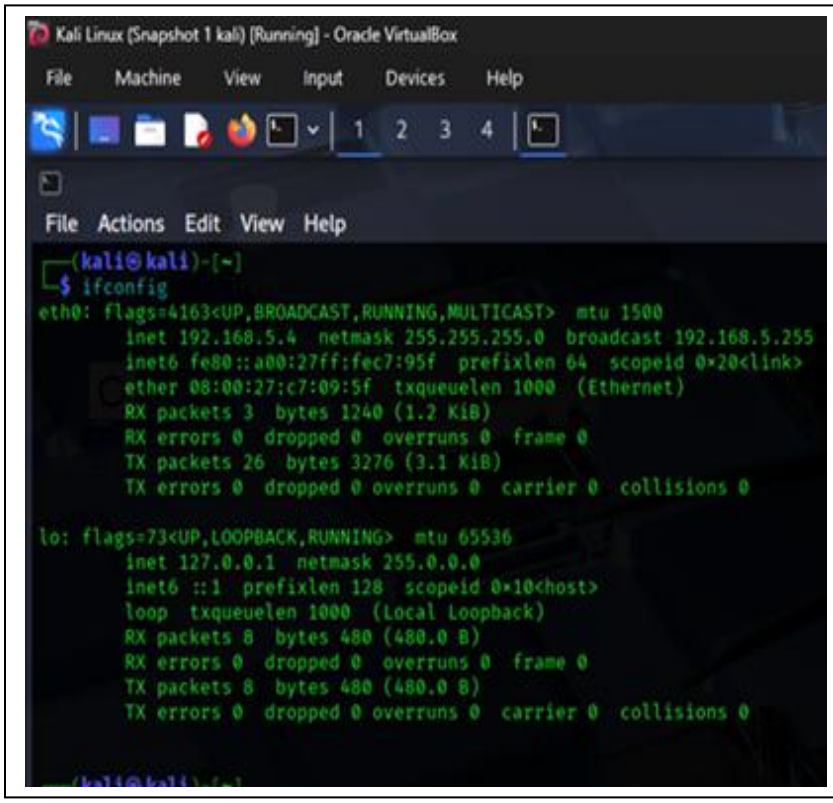
The objective of this lab is to practically demonstrate how the Suricata Intrusion Detection System (IDS) can detect and log a Denial-of-Service (DoS) attack. This lab will explain how to configure Suricata, understand network traffic, simulate a DoS attack using hping3, analyze logs, and understand IDS rule-based detection.

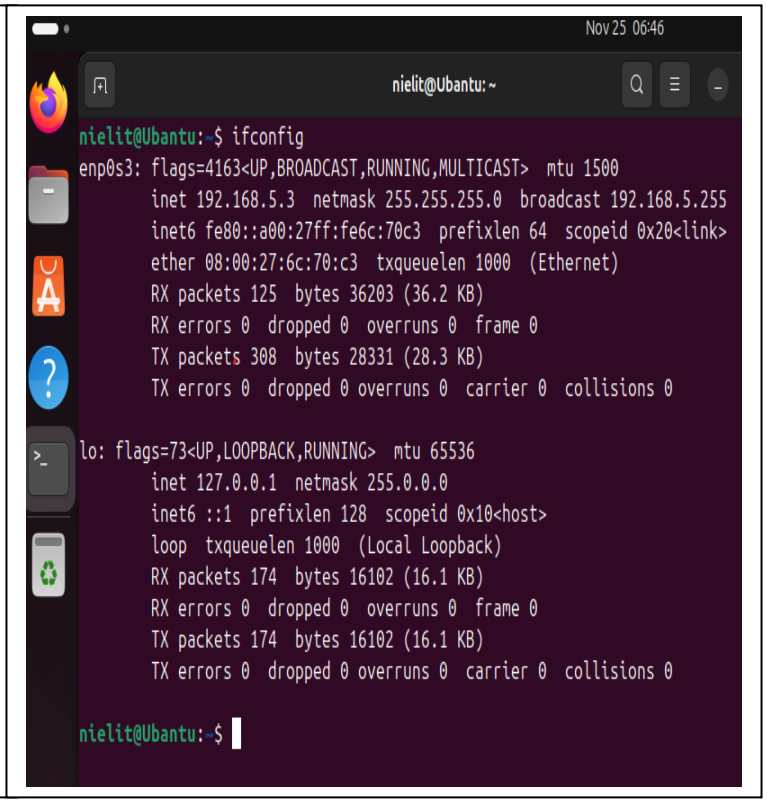
,

- Understand DoS attacks and their characteristics.
- Configure and run Suricata IDS.
- Generate different kinds of attack traffic.
- Analyze Suricata alert logs.
- Create custom rules for detecting anomalous traffic.

2. Lab topology and prerequisites

- **Attacker:** Kali Linux VM (IP example: 192.168.5.4)
- **Victim / IDS host:** Ubuntu VM running Suricata (IP example: 192.168.5.3)
- Both VMs on same isolated network (e.g., host-only or internal VM network)
- Root or sudo access on both machines
- Internet access optional for package installs (or use local package cache)





Software to install for Detecting DoS Attack using Kali Linux

- Kali- hping3, nmap
- Ubuntu- suricata, ufw, tcpdump, jq

Commands (run as sudo) –

sudo apt update && sudo apt install -y suricata ufw tcpdump jq

This command is used to update the system's package information and install essential security and networking tools on a Debian-based Linux system such as Kali Linux or Ubuntu. It update package repositories and installing Suricata, UFW, tcpdump, and jq to enable intrusion detection, firewall management, packet capture, and structured log analysis.

,

sudo
Executes the command with administrator (root) privileges.
Required for system-level package updates and installations.

apt update
Refreshes the local package index by fetching the latest package lists from configured repositories.
Ensures that the most recent and secure versions of software are installed.

&&
A logical operator that ensures the second command runs only if the first command succeeds.
Prevents installation if the package list update fails.
apt install -y suricata ufw tcpdump jq
Installs multiple tools in one command.

Suricata
An open-source Intrusion Detection and Prevention System (IDS/IPS).
Used to monitor network traffic and detect attacks such as DoS, scans, and malware.

UFW (Uncomplicated Firewall)

A user-friendly firewall interface.
Helps manage firewall rules to control incoming and outgoing network traffic.

tcpdump
A powerful packet capture and network traffic analysis tool.
Used for inspecting live traffic and troubleshooting network issues.

jq
A lightweight command-line JSON processor.
Used to parse and analyze JSON logs such as Suricata's eve.json.

-y flag
Automatically answers "yes" to installation prompts.
Enables non-interactive installation, useful for scripting and automation.

sudo apt install -y hping3 nmap (on Kali)
This command prepares the Kali Linux system with tools required for network reconnaissance and traffic generation, which are essential for testing intrusion detection systems like Suricata and simulating real-world ne This command installs network scanning and packet-generation tools on Kali Linux for security testing and network analysis.

sudo
Runs the command with root (administrator) privileges.
Required because installing system packages modifies core system files.

apt install
Uses Kali Linux's APT package manager to install software from official repositories.

,

-y
Automatically confirms all installation prompts.
Enables unattended or scripted installation.

Installed Tools

◈ **hping3**

A command-line packet crafting and traffic generation tool.
Used to send custom TCP/IP packets. Commonly used for:
-Simulating DoS/DDoS attacks
-Firewall testing
-TCP SYN flood testing
-Packet-level network analysis

◈ **nmap**

A powerful **network scanning and reconnaissance tool**. Used to discover:
-Live hosts on a network
-Open ports and running services
-Operating system and service versions
Widely used in **penetration testing and vulnerability assessment**.

2.1 Technical Requirements

Two Machines (Virtual)

- One for Attack (Kali Linux)
- One for Victim/IDS (Ubuntu or Kali Linux)
- 2 GB Minimum RAM per machine
- Working network configuration (Host-only / Bridged / NAT)

2.2 Software Requirements

- Suricata IDS installed on the victim machine
- hping3 installed on the attacker machine
- Basic tools like ping, ip, ifconfig

2.3 Basic Knowledge Required

Students should have basic understanding of -

- Linux commands
- TCP/IP networking
- IDS/IPS concepts
- Log analysis

3. Lab Topology

3.1 Network Diagram (Conceptual)

```
(Attacker Machine: Kali Linux)
       IP: 192.168.5.4
                |
       ----------------
       |              |
Local Network / Virtual Switch
       |              |
       ----------------
                |
(Victim + IDS: Ubuntu with Suricata)
       IP: 192.168.5.3
```

3.2 Traffic Flow

- Attacker sends high-rate SYN traffic - Victim's Port 80
- Suricata monitors network interface - Detects anomalies
- Logs generated in /var/log/suricata/

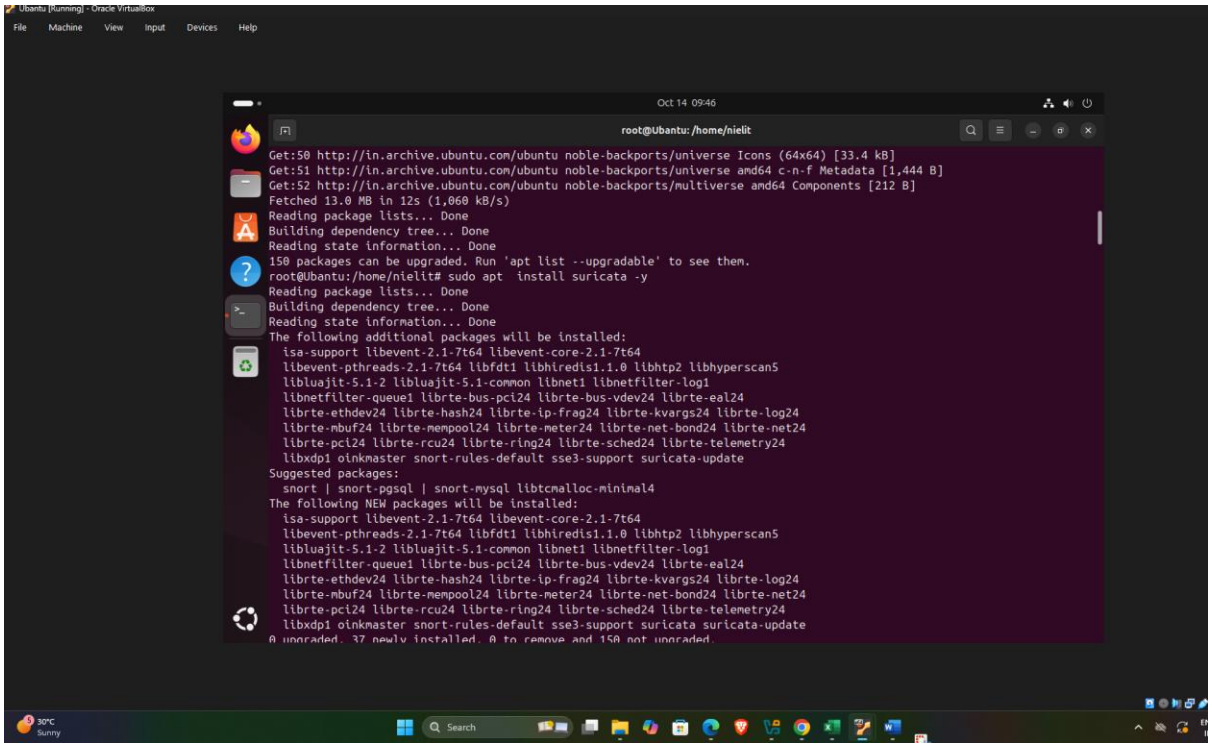
4. Installation & Configuration of Suricata

4.1 Installing Suricata (On the victim machine**)**

sudo apt update
sudo apt install suricata -y

Verify installation-

- **Sudo systemctl restart suricata**
- **Sudo systemctl status suricata**
- **Suricata update –version**



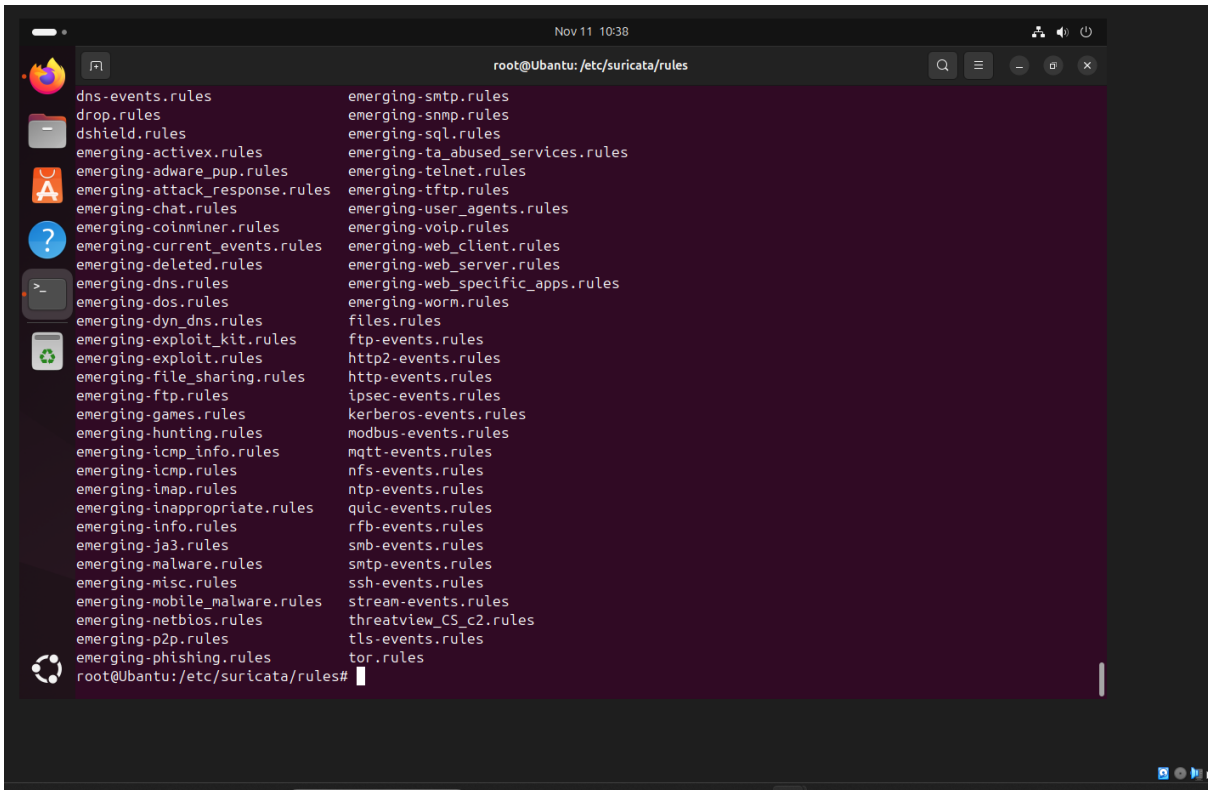


Fig. Installation of Suricata

,

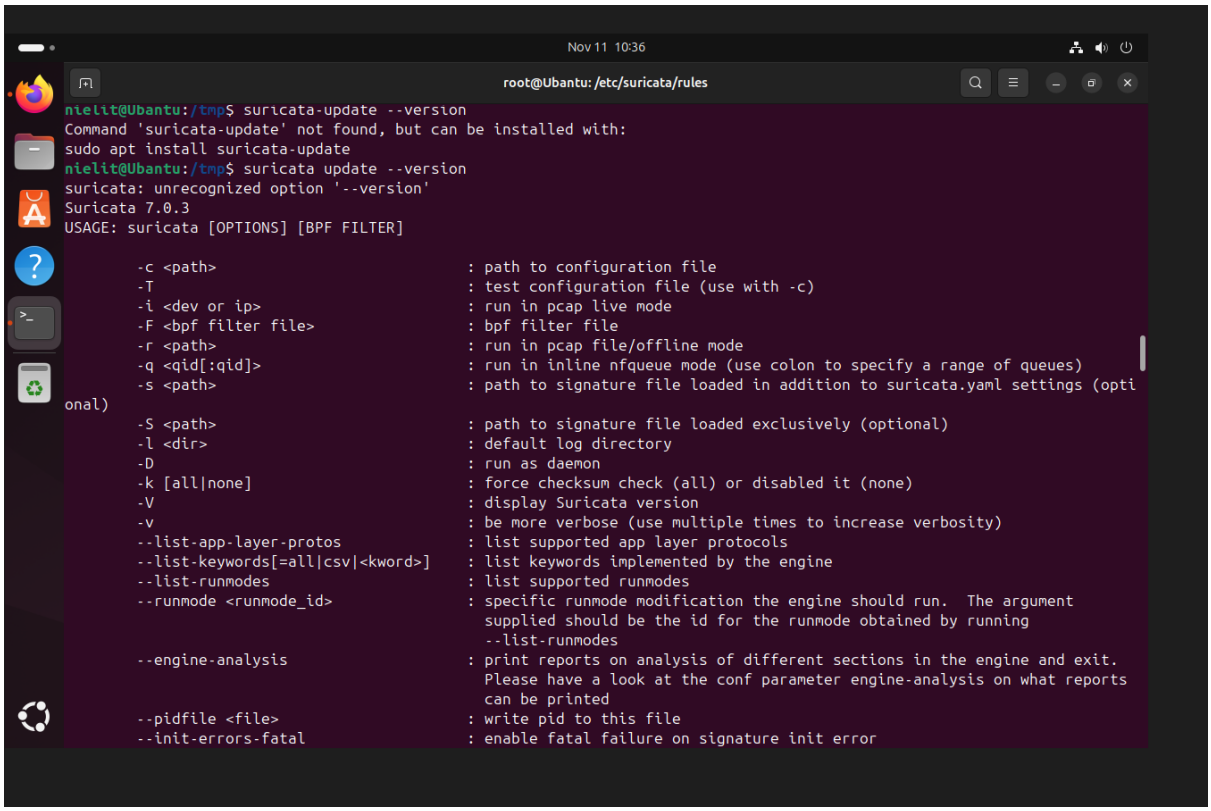


Fig. Suricata Version

2. Identifying Network Interface

Use command

- ifconfig

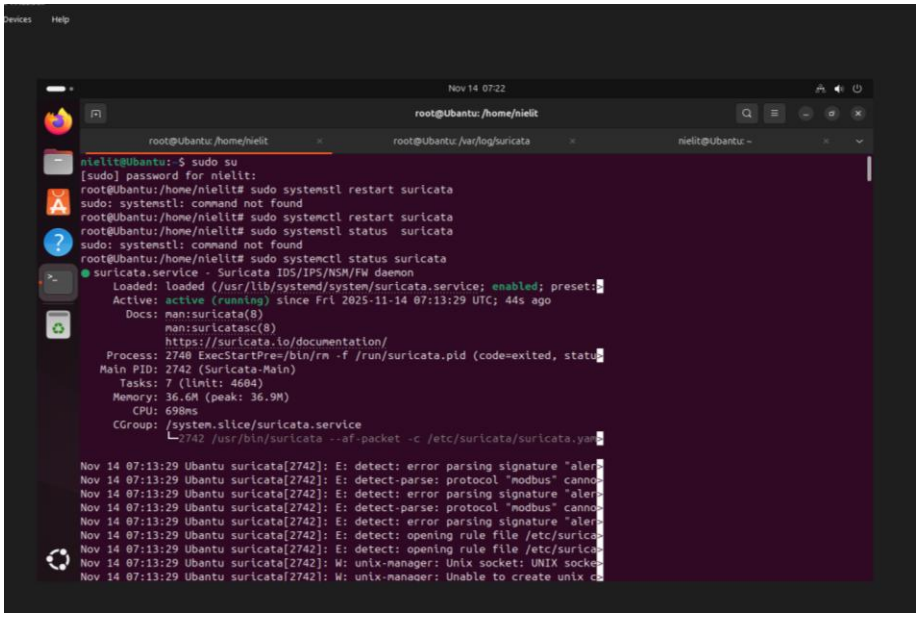
  – To get Ethernet information

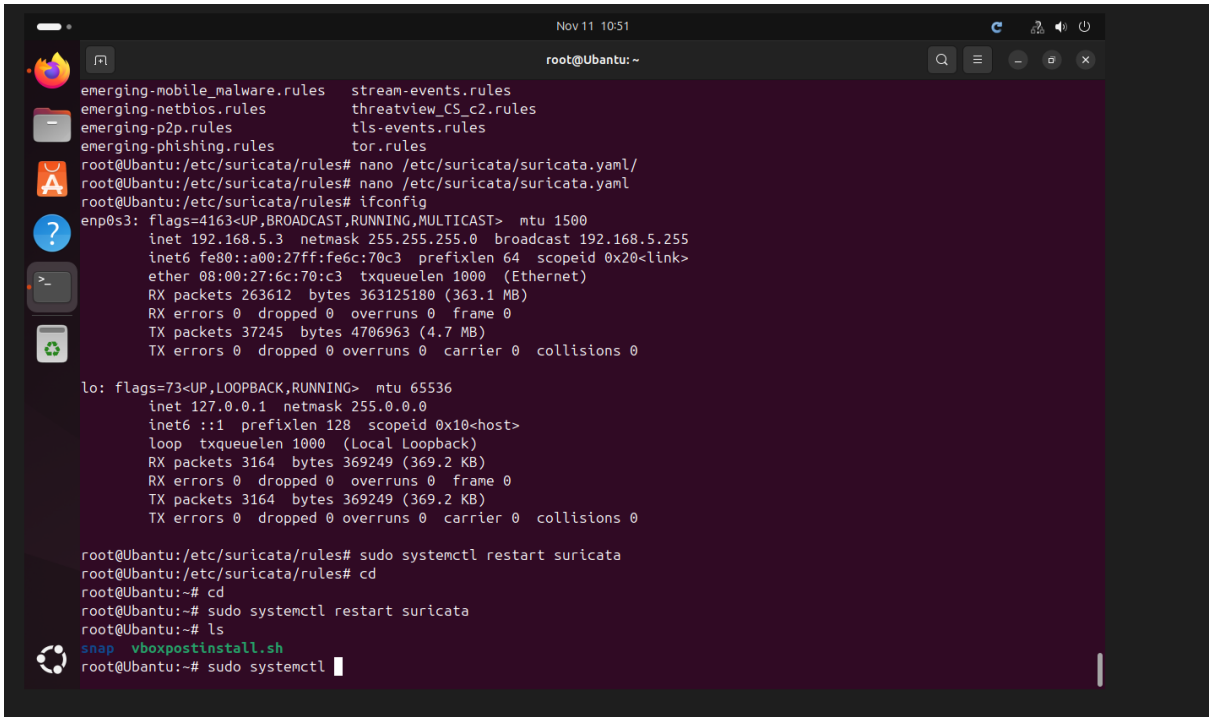
- ip a

  – It shows **Interface names** (lo, eth0, wlan0, enp0s3, etc.), **State** (UP/DOWN), MAC address, **IPv4 address** (e.g., 192.168.5.3/24), IPv6 address, MTU size.

-Note the correct interface (e.g., enp0s3, eth0). This will be important for running Suricata.

,


Devices Help
Nov 14 07:22
root@Ubantu: /home/nielit
root@Ubantu: /home/nielit
root@Ubantu: /var/log/suricata
nielit@Ubantu: ~
nielit@Ubantu: $ sudo su
[sudo] password for nielit:
root@Ubantu:/home/nielit# sudo systemstl restart suricata
sudo: systemstl: command not found
root@Ubantu:/home/nielit# sudo systemctl restart suricata
root@Ubantu:/home/nielit# sudo systemstl status suricata
sudo: systemstl: command not found
root@Ubantu:/home/nielit# sudo systemctl status suricata
● suricata.service - Suricata IDS/IPS/NSM/FW daemon
Loaded: loaded (/usr/lib/systemd/system/suricata.service; enabled; preset:
Active: active (running) since Fri 2025-11-14 07:13:29 UTC; 44s ago
Docs: man:suricata(8)
man:suricatasc(8)
https://suricata.io/documentation/
Process: 2740 ExecStartPre=/bin/rm -f /run/suricata.pid (code=exited, statu
Main PID: 2742 (Suricata-Main)
Tasks: 7 (limit: 4604)
Memory: 36.6M (peak: 36.9M)
CPU: 698ms
CGroup: /system.slice/suricata.service
└2742 /usr/bin/suricata --af-packet -c /etc/suricata/suricata.yam
Nov 14 07:13:29 Ubantu suricata[2742]: E: detect: error parsing signature "aler
Nov 14 07:13:29 Ubantu suricata[2742]: E: detect-parse: protocol "modbus" canno
Nov 14 07:13:29 Ubantu suricata[2742]: E: detect: error parsing signature "aler
Nov 14 07:13:29 Ubantu suricata[2742]: E: detect-parse: protocol "modbus" canno
Nov 14 07:13:29 Ubantu suricata[2742]: E: detect: error parsing signature "aler
Nov 14 07:13:29 Ubantu suricata[2742]: E: detect: opening rule file /etc/surica
Nov 14 07:13:29 Ubantu suricata[2742]: E: detect: opening rule file /etc/surica
Nov 14 07:13:29 Ubantu suricata[2742]: W: unix-manager: Unix socket: UNIX socke
Nov 14 07:13:29 Ubantu suricata[2742]: W: unix-manager: Unable to create unix c


Nov 11 10:51
root@Ubantu: ~
emerging-mobile_malware.rules stream-events.rules
emerging-netbios.rules threatview_CS_c2.rules
emerging-p2p.rules tls-events.rules
emerging-phishing.rules tor.rules
root@Ubantu:/etc/suricata/rules# nano /etc/suricata/suricata.yaml/
root@Ubantu:/etc/suricata/rules# nano /etc/suricata/suricata.yaml
root@Ubantu:/etc/suricata/rules# ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
inet 192.168.5.3 netmask 255.255.255.0 broadcast 192.168.5.255
inet6 fe80::a00:27ff:fe6c:70c3 prefixlen 64 scopeid 0x20<link>
ether 08:00:27:6c:70:c3 txqueuelen 1000 (Ethernet)
RX packets 263612 bytes 363125180 (363.1 MB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 37245 bytes 4706963 (4.7 MB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0
inet6 ::1 prefixlen 128 scopeid 0x10<host>
loop txqueuelen 1000 (Local Loopback)
RX packets 3164 bytes 369249 (369.2 KB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 3164 bytes 369249 (369.2 KB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
root@Ubantu:/etc/suricata/rules# sudo systemctl restart suricata
root@Ubantu:/etc/suricata/rules# cd
root@Ubantu:~# cd
root@Ubantu:~# sudo systemctl restart suricata
root@Ubantu:~# ls
snap vboxpostinstall.sh
root@Ubantu:~# sudo systemctl

```
emerging-mobile_malware.rules   stream-events.rules
emerging-netbios.rules          threatview_CS_c2.rules
emerging-p2p.rules              tls-events.rules
emerging-phishing.rules         tor.rules
root@Ubantu:/etc/suricata/rules# nano /etc/suricata/suricata.yaml/
root@Ubantu:/etc/suricata/rules# nano /etc/suricata/suricata.yaml
root@Ubantu:/etc/suricata/rules# ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.5.3  netmask 255.255.255.0  broadcast 192.168.5.255
        inet6 fe80::a00:27ff:fe6c:70c3  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:6c:70:c3  txqueuelen 1000  (Ethernet)
        RX packets 263612  bytes 363125180 (363.1 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 37245  bytes 4706963 (4.7 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 3164  bytes 369249 (369.2 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 3164  bytes 369249 (369.2 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

root@Ubantu:/etc/suricata/rules# sudo systemctl restart suricata
root@Ubantu:/etc/suricata/rules# cd
root@Ubantu:~# cd
root@Ubantu:~# sudo systemctl restart suricata
root@Ubantu:~# ls
snap  vboxpostinstall.sh
root@Ubantu:~# sudo systemctl
```

```
11/11/2025 -- 07:48:59 - <Info> -- Done.
nielit@Ubantu:/tmp$ ls /var/lib/suricata/rules/
ls: cannot access '/var/lib/suricata/rules/': Permission denied
nielit@Ubantu:/tmp$ cd /tmp/ && curl -LO https://rules.emergingthreats.net/open/suricata-6.0.8/emerging.rules.tar.gz
sudo tar -xvzf emerging.rules.tar.gz && sudo mv rules/*.rules /etc/suricata/rules/
sudo chmod 640 /etc/suricata/rules/*.rules
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 5062k  100 5062k    0     0   417k      0  0:00:12  0:00:12 --:--:--  429k
rules/
rules/BSD-License.txt
rules/LICENSE
rules/botcc.portgrouped.rules
rules/botcc.rules
rules/ciarmy.rules
rules/classification.config
rules/compromised-ips.txt
rules/compromised.rules
rules/drop.rules
rules/dshield.rules
rules/emerging-activex.rules
rules/emerging-adware_pup.rules
rules/emerging-attack_response.rules
rules/emerging-chat.rules
rules/emerging-coinminer.rules
rules/emerging-current_events.rules
rules/emerging-deleted.rules
rules/emerging-dns.rules
rules/emerging-dos.rules
rules/emerging-exploit.rules
rules/emerging-exploit_kit.rules
rules/emerging-ftp.rules
rules/emerging-games.rules
```

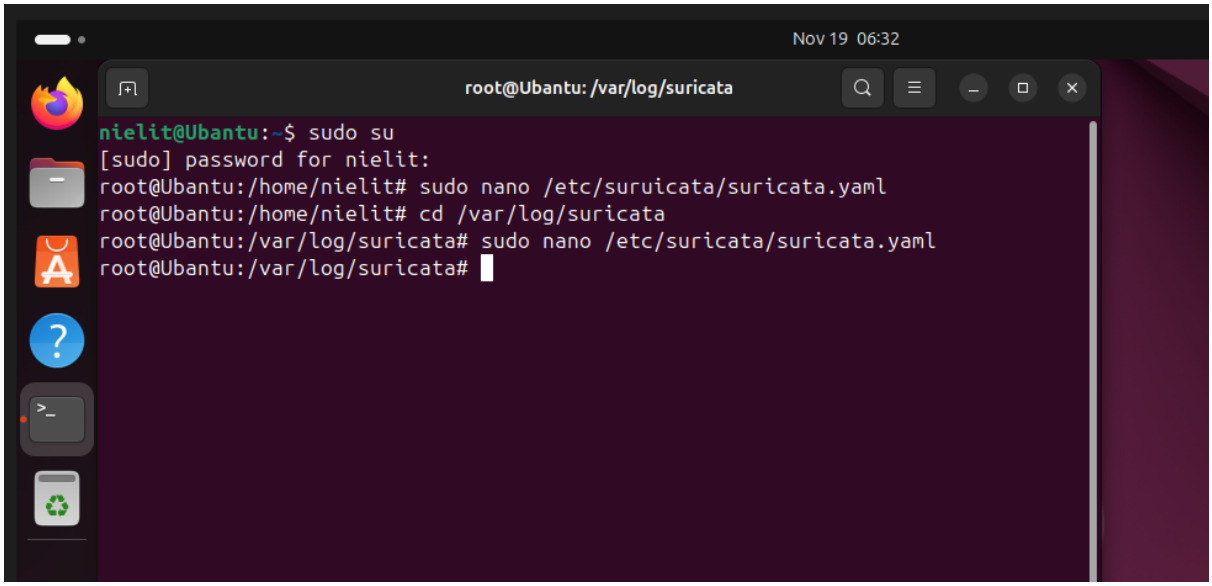
,

Suricata Rules-

4.3 Configuring Suricata YAML File

Open the configuration file:

- cd /var/log/suricata
- sudo nano /etc/suricata/suricata.yaml



Important fields to configure:

4.3.1 HOME_NET

Replace your Home Net with victim machine (Ubuntu) IP Address

- HOME_NET: “[192.168.5.3]”

This defines protected network.

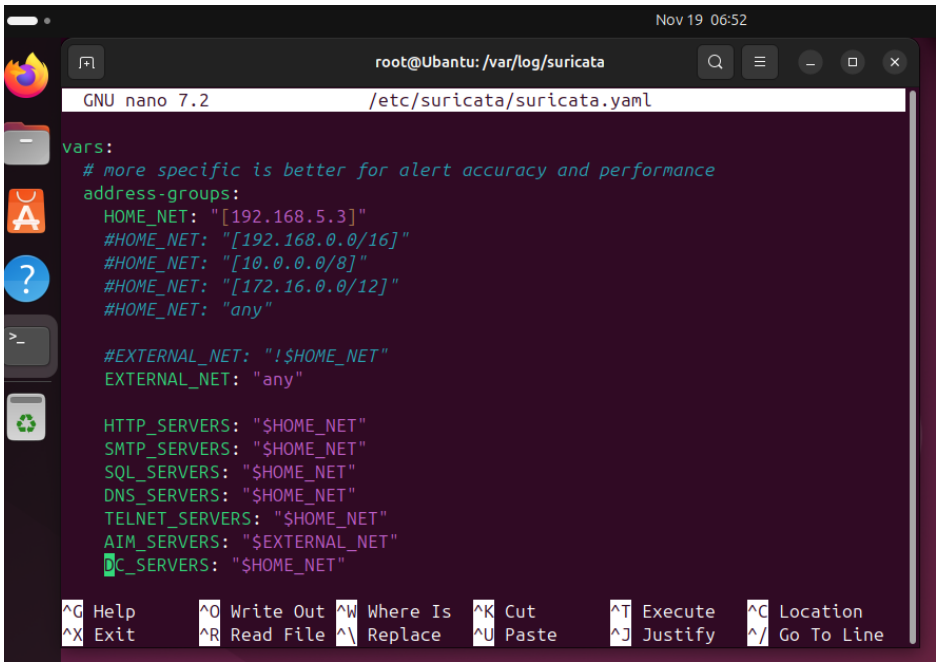
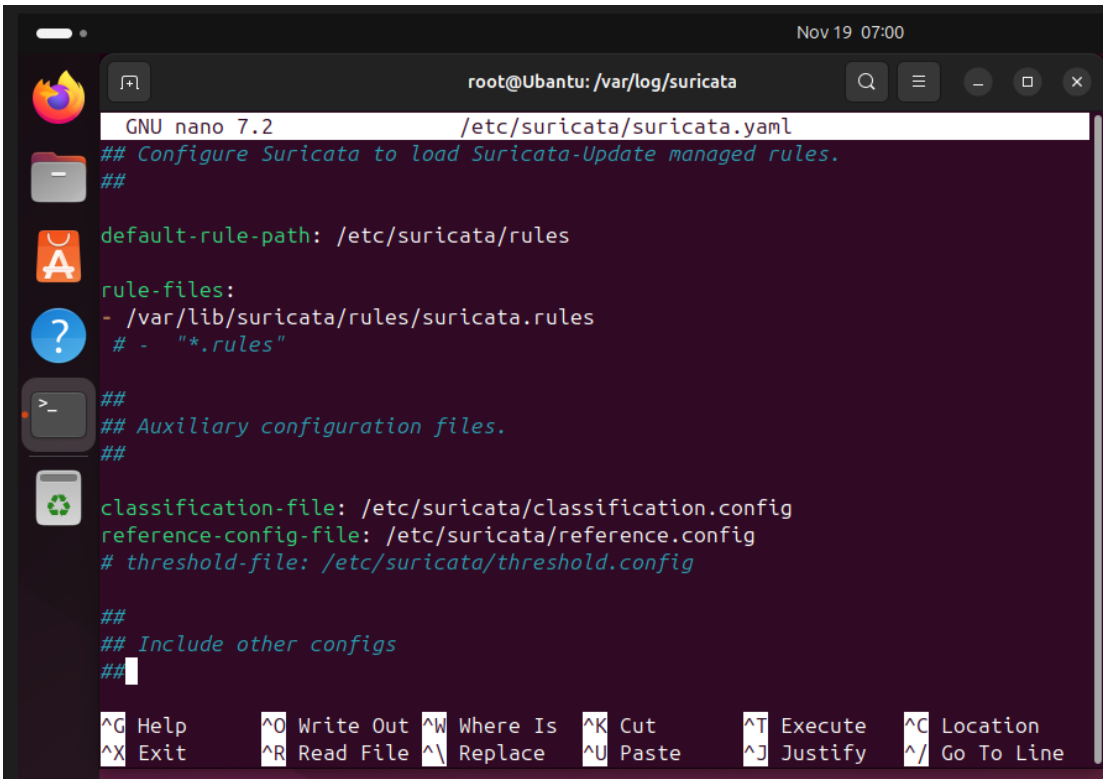


Fig. Home Net Configuration

4.3.2 Rule Path Verification

Ensure these directories exist in rule file Suricata yaml file.

- /var/lib/suricata/rules/
- /etc/suricata/rules/
- - /var/lib/suricata/rules/suricata.rules

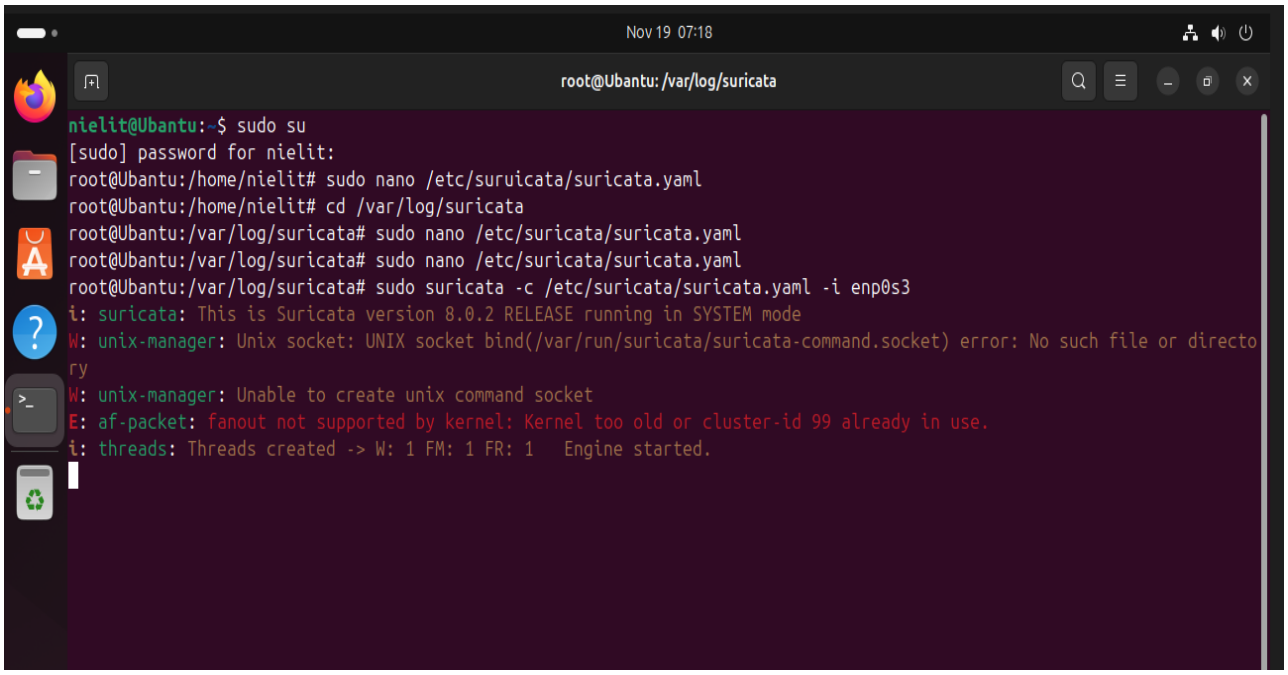


,

4.4 Starting Suricata in IDS Mode

Stop the service to run manually
sudo systemctl stop suricata

Run IDS mode
sudo suricata -c /etc/suricata/suricata.yaml -i enp0s3

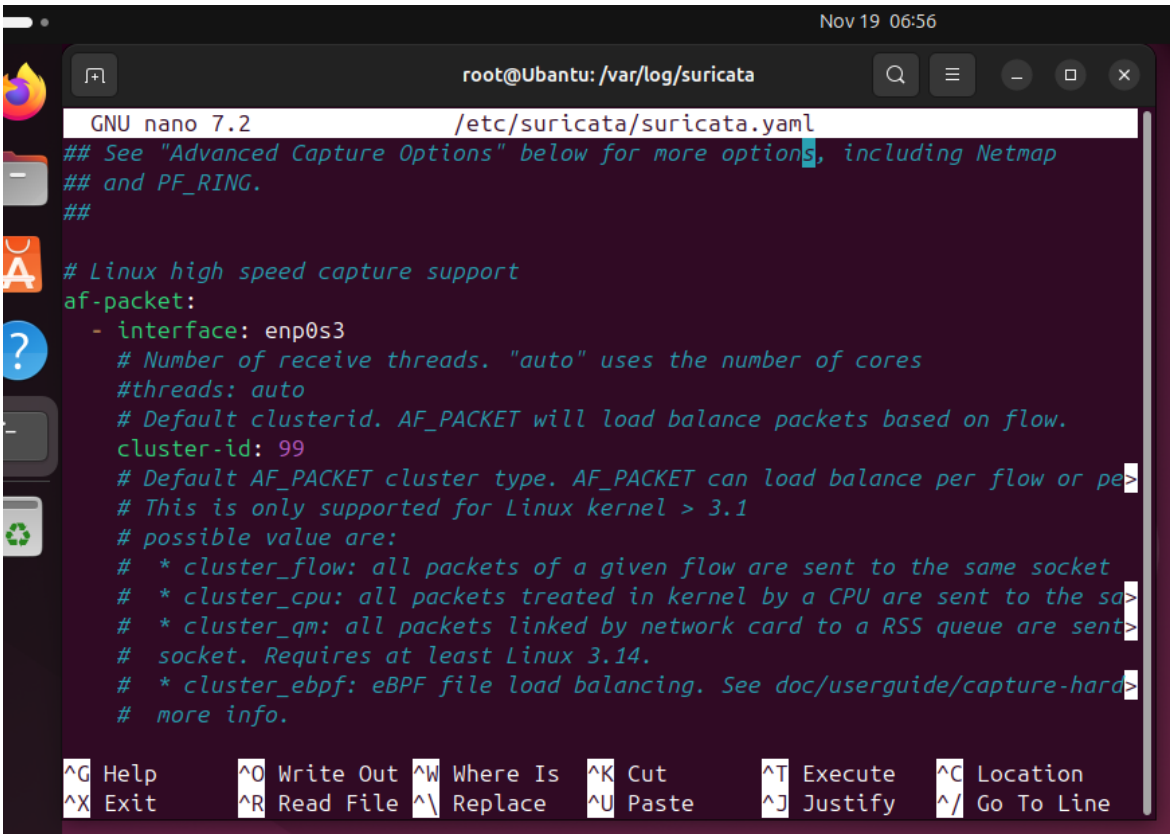


,

- **Replace enp0s3 with your interface**

  af -packet:

  -interface: enps03



- **Restart Suricata**

Command- **sudo systemctl restart suricata**

This command is used to restart the Suricata Intrusion Detection and Prevention System (IDS/IPS) service on a Linux system.
Sudo - Executes the command with administrator (root) privileges, which are required to control system services.
Systemctl - A system utility used to manage system services under systemd, such as starting, stopping, or restarting services.
Restart - Stops the Suricata service if it is currently running and then starts it again. This ensures that any recent configuration or rule changes are applied.
Suricata - Refers to the Suricata IDS/IPS service responsible for monitoring network traffic and detecting malicious activities, including DDoS attacks.

Purpose in DDoS Detection Project

In this DDoS attack detection project, this command is executed after:

- Updating or modifying Suricata rule files
- Changing network interface settings
- Enabling custom DDoS detection rules

Restarting Suricata ensures that:

,

- The latest DDoS detection rules are loaded
- Suricata starts actively monitoring traffic
- The IDS generates alerts in log files (such as `eve.json`) when suspicious or high-volume traffic patterns are detected

5. Verifying Suricata Operation

5.1 Live Log Monitoring

Command - **sudo tail -f /var/log/suricata/fast.log**

Tail
tail is a Linux command that shows the last lines of a file.
Normally it prints only the last 10 lines.

-f (Follow mode)
`-f` stands for follow. This tells `tail` to continuously monitor the file and display new lines in real time as they are added. Suricata will start printing real-time detection information.

5.1 On Kali Machine (Attacker)

Install nmap by using the following command-

sudo apt update

It is executed before installing or upgrading tools like Suricata, tcpdump, or other network analysis utilities. After running this command, the system's package list is refreshed. The latest Suricata packages and rule updates become available. The environment is prepared for reliable installation and configuration of the IDS.

sudo apt install nmap

nmap – version
Scan all common ports –
sudo nmap -sS 192.168.5.3

,

```
┌──(kali㉿kali)-[~]
└─$ sudo apt install nmap -y   # if needed (use NAT temporarily)
sudo nmap -T4 -p- 192.168.56.11

nmap is already the newest version (7.95+dfsg-3kali1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1218
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 15:19 IST
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 1.51 seconds

┌──(kali㉿kali)-[~]
└─$ sudo apt install nmap -y   # if needed (use NAT temporarily)
sudo nmap -T4 -p- 192.168.56.102
nmap is already the newest version (7.95+dfsg-3kali1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1218
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-30 15:22 IST
Nmap scan report for 192.168.56.102
Host is up (0.0019s latency).
All 65535 scanned ports on 192.168.56.102 are in ignored states.
Not shown: 65535 closed tcp ports (reset)
MAC Address: 08:00:27:6C:70:C3 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 26.48 seconds
```

- **Scan specific port-** example scan ports 80 and 443
  sudo nmap -sS -p 80,443 192.168.5.3

- **Scan a range of port-** scan ports 1 to 1000
  sudo nmap -sS -p 1-1000 192.168.5.3

- **Scan all 65,535 ports (for DoS/IDS project testing)-**
  sudo nmap -sS -p- 192.168.5.3

- **Service and version detection (useful for Suricata alerts)**
  sudo nmap -sS -sV 192.168.5.3

Why sudo is required

- -sS (SYN scan) uses raw packets.

,

- Raw packet scanning requires root privileges.

  Without sudo, Nmap will fail or downgrade the scan type.

```
┌──(root㉿kali)-[/home/kali]
└─# sudo nmap -sS 192.168.5.3

Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-16 11:15 IST
Nmap scan report for 192.168.5.3
Host is up (0.0022s latency).
Not shown: 998 closed tcp ports (reset)
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http
MAC Address: 08:00:27:6C:70:C3 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 11.38 seconds

┌──(root㉿kali)-[/home/kali]
└─# sudo nmap -sS -p 80,443 192.168.5.3

Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-16 11:15 IST
Nmap scan report for 192.168.5.3
Host is up (0.0010s latency).

PORT    STATE  SERVICE
80/tcp  open   http
443/tcp closed https
MAC Address: 08:00:27:6C:70:C3 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 7.64 seconds
```

,

```
PORT    STATE  SERVICE
80/tcp  open   http
443/tcp closed https
MAC Address: 08:00:27:6C:70:C3 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 7.64 seconds

┌──(root㉿kali)-[/home/kali]
└─# sudo nmap -sS -p 1-1000 192.168.5.3

Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-16 11:16 IST
Nmap scan report for 192.168.5.3
Host is up (0.0045s latency).
Not shown: 998 closed tcp ports (reset)
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http
MAC Address: 08:00:27:6C:70:C3 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 11.38 seconds

┌──(root㉿kali)-[/home/kali]
└─# sudo nmap -sS -sV 192.168.5.3

Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-16 11:16 IST
Nmap scan report for 192.168.5.3
Host is up (0.00063s latency).
Not shown: 998 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux; protocol 2.0)
80/tcp open  http    Apache httpd 2.4.58 ((Ubuntu))
MAC Address: 08:00:27:6C:70:C3 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 14.44 seconds

┌──(root㉿kali)-[/home/kali]
└─# sudo hping3 -S --flood -V -p 80 192.168.5.3
using eth0, addr: 192.168.5.4, MTU: 1500
HPING 192.168.5.3 (eth0 192.168.5.3): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
^C
--- 192.168.5.3 hping statistic ---
2455956 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms

┌──(root㉿kali)-[/home/kali]
└─#
```

6. Launching a DoS Attack Using hping3

6.1 Installing hping3 on Kali attacker

- sudo apt install hping3 -y

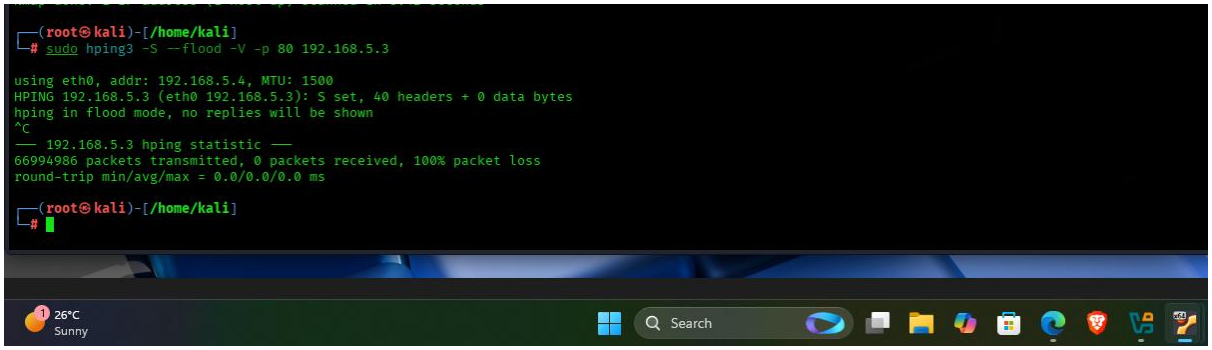
```
┌──(kali㉿kali)-[~]
└─$ sudo apt install hping3 -y
hping3 is already the newest version (3.a2.ds2-11~kali1).
hping3 set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1218

┌──(kali㉿kali)-[~]
```

,

6.2 Types of Attacks You Can Perform

6.2.1 SYN Flood Attack

- sudo hping3 -S --flood -V -p 80 192.168.5.3



6.2.2 UDP Flood Attack

- sudo hping3 --udp --flood -p 53 192.168.5.3

6.2.3 ICMP Flood Attack

- sudo hping3 --icmp --flood 192.168.5.3

6.3 Explanation of Important Flags

- -S → SYN packet
- --flood → Maximum speed
- -V → Verbose output
- -p → Target port
- --udp → UDP mode
- --icmp → ICMP mode

3. Why '0 packets received' appears

When use `--flood`, hping

- does not wait for responses
- ignores all incoming packets
- always reports 0 received

This is normal for flood mode.

hping3 warns as- hping in flood mode, no replies will be shown.

This does not mean-

- the victim is down
- the packets were blocked
- the attack failed

,

It only means- **hping is not showing responses because flood mode suppresses them.**

4. What happened on the victim (192.168.5.3)

On the Ubuntu victim:

- A very high number of SYN packets were received
- System get slow
- Port 80 becomes busy
- TCP backlog queue fills
- Services become unreachable
- Kernel log-

7. Monitoring Suricata Logs During Attack

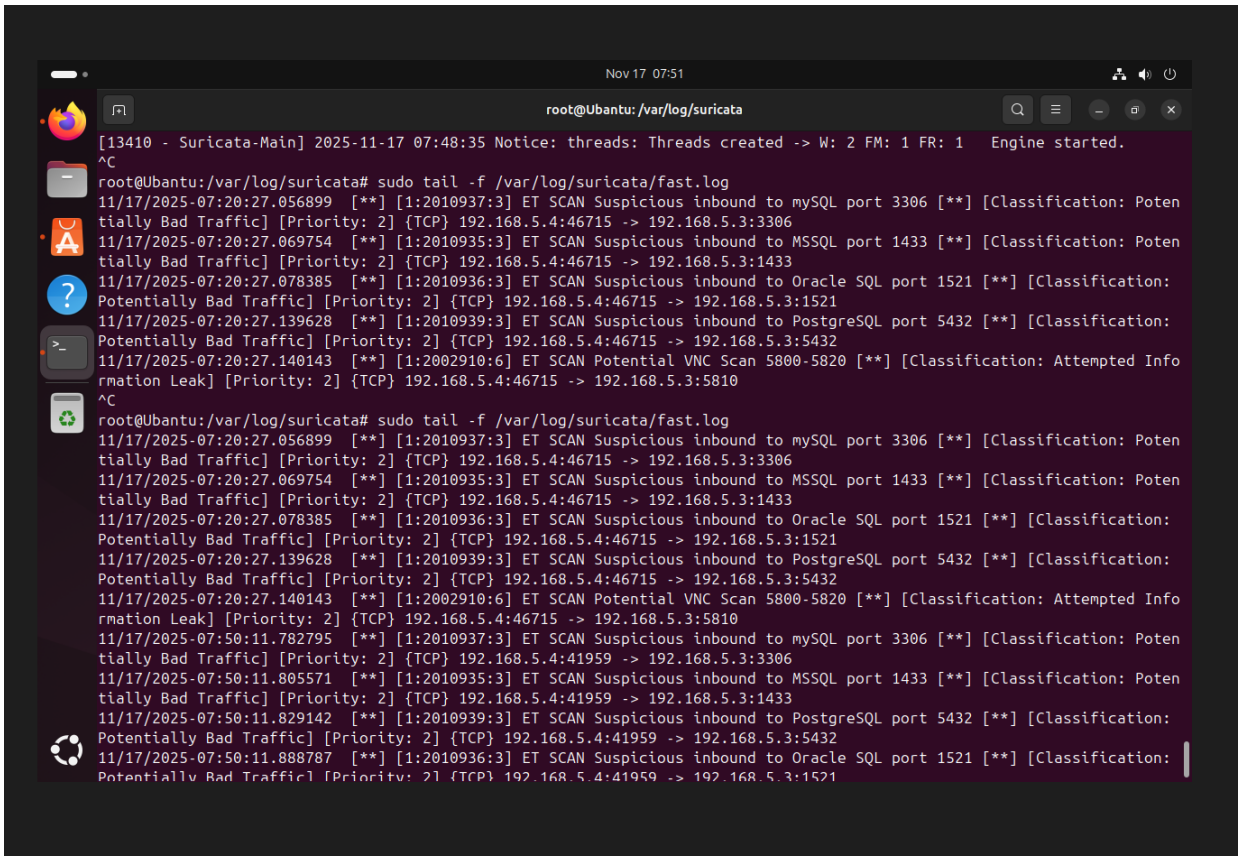
7.1 fast.log Monitoring

On victim machine:

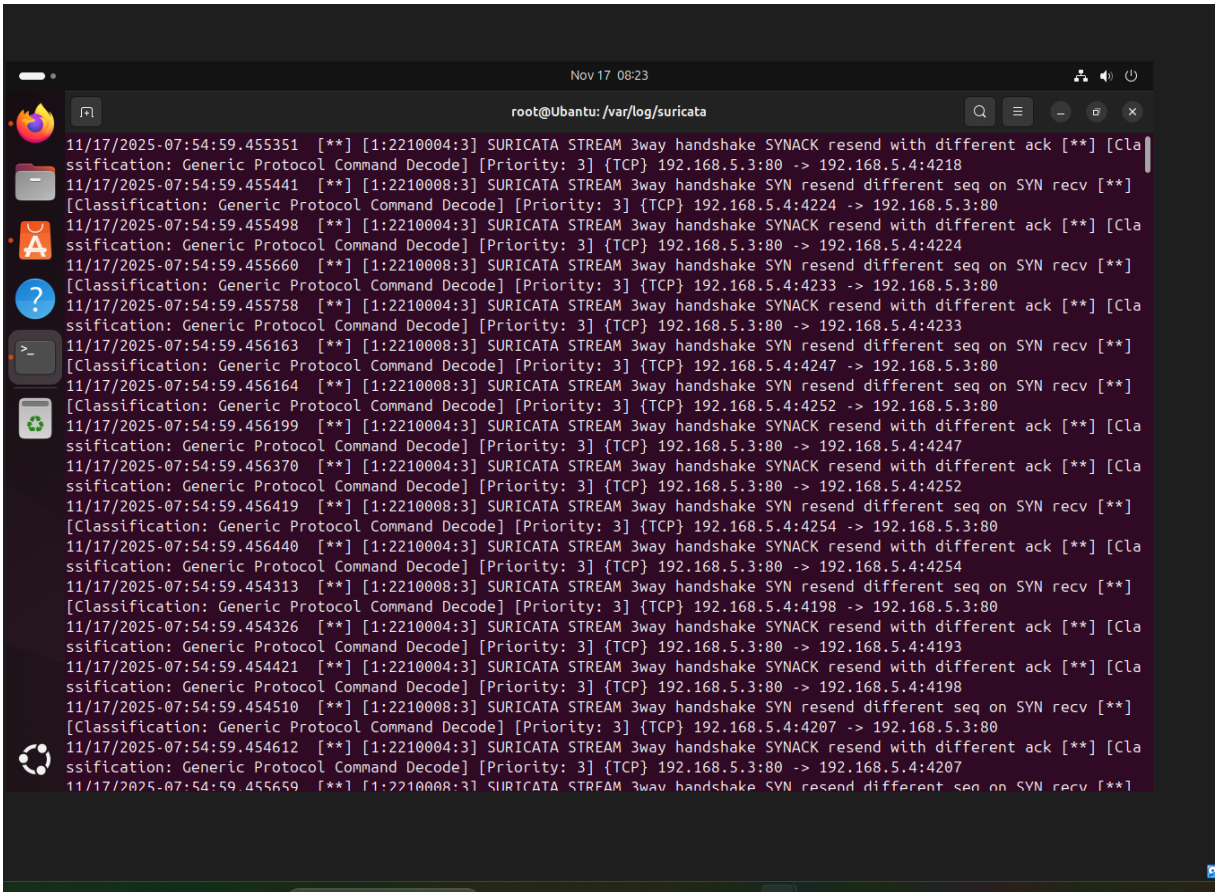
- sudo tail -f /var/log/suricata/fast.log

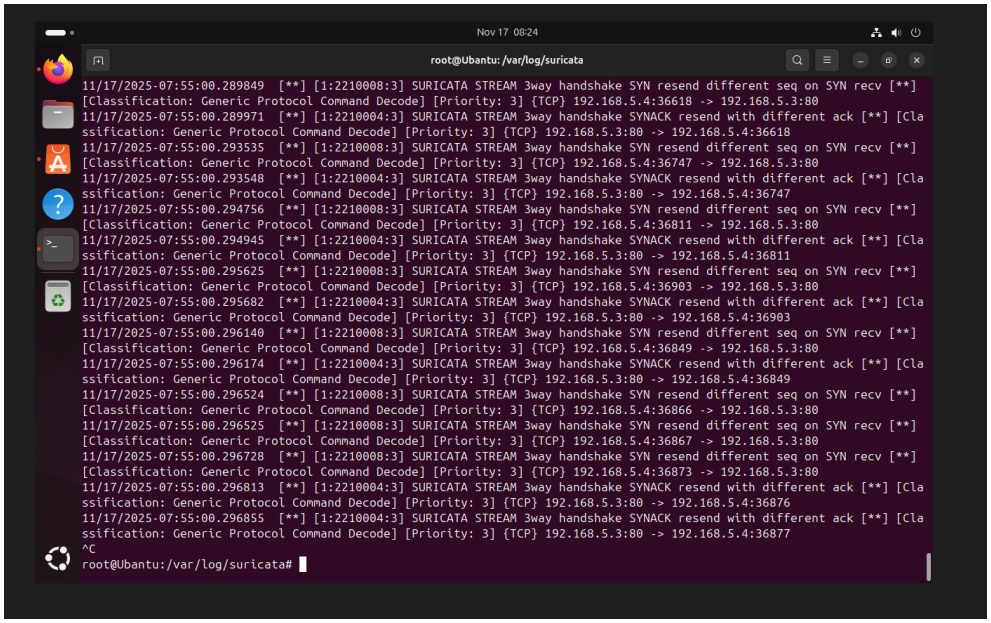
Suricata on the victim should log this as a SYN Flood attack

Suricata IDS on Ubuntu shows SYN alerts-



,


Nov 17 08:23
root@Ubantu: /var/log/suricata
11/17/2025-07:54:59.455351 [**] [1:2210004:3] SURICATA STREAM 3way handshake SYNACK resend with different ack [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.3:80 -> 192.168.5.4:4218
11/17/2025-07:54:59.455441 [**] [1:2210008:3] SURICATA STREAM 3way handshake SYN resend different seq on SYN recv [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.4:4224 -> 192.168.5.3:80
11/17/2025-07:54:59.455498 [**] [1:2210004:3] SURICATA STREAM 3way handshake SYNACK resend with different ack [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.3:80 -> 192.168.5.4:4224
11/17/2025-07:54:59.455660 [**] [1:2210008:3] SURICATA STREAM 3way handshake SYN resend different seq on SYN recv [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.4:4233 -> 192.168.5.3:80
11/17/2025-07:54:59.455758 [**] [1:2210004:3] SURICATA STREAM 3way handshake SYNACK resend with different ack [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.3:80 -> 192.168.5.4:4233
11/17/2025-07:54:59.456163 [**] [1:2210008:3] SURICATA STREAM 3way handshake SYN resend different seq on SYN recv [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.4:4247 -> 192.168.5.3:80
11/17/2025-07:54:59.456164 [**] [1:2210008:3] SURICATA STREAM 3way handshake SYN resend different seq on SYN recv [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.4:4252 -> 192.168.5.3:80
11/17/2025-07:54:59.456199 [**] [1:2210004:3] SURICATA STREAM 3way handshake SYNACK resend with different ack [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.3:80 -> 192.168.5.4:4247
11/17/2025-07:54:59.456370 [**] [1:2210004:3] SURICATA STREAM 3way handshake SYNACK resend with different ack [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.3:80 -> 192.168.5.4:4252
11/17/2025-07:54:59.456419 [**] [1:2210008:3] SURICATA STREAM 3way handshake SYN resend different seq on SYN recv [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.4:4254 -> 192.168.5.3:80
11/17/2025-07:54:59.456440 [**] [1:2210004:3] SURICATA STREAM 3way handshake SYNACK resend with different ack [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.3:80 -> 192.168.5.4:4254
11/17/2025-07:54:59.454313 [**] [1:2210008:3] SURICATA STREAM 3way handshake SYN resend different seq on SYN recv [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.4:4198 -> 192.168.5.3:80
11/17/2025-07:54:59.454326 [**] [1:2210004:3] SURICATA STREAM 3way handshake SYNACK resend with different ack [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.3:80 -> 192.168.5.4:4193
11/17/2025-07:54:59.454421 [**] [1:2210004:3] SURICATA STREAM 3way handshake SYNACK resend with different ack [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.3:80 -> 192.168.5.4:4198
11/17/2025-07:54:59.454510 [**] [1:2210008:3] SURICATA STREAM 3way handshake SYN resend different seq on SYN recv [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.4:4207 -> 192.168.5.3:80
11/17/2025-07:54:59.454612 [**] [1:2210004:3] SURICATA STREAM 3way handshake SYNACK resend with different ack [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.3:80 -> 192.168.5.4:4207


Nov 17 08:24
root@Ubantu: /var/log/suricata
11/17/2025-07:55:00.289849 [**] [1:2210008:3] SURICATA STREAM 3way handshake SYN resend different seq on SYN recv [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.4:36618 -> 192.168.5.3:80
11/17/2025-07:55:00.289971 [**] [1:2210004:3] SURICATA STREAM 3way handshake SYNACK resend with different ack [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.3:80 -> 192.168.5.4:36618
11/17/2025-07:55:00.293535 [**] [1:2210008:3] SURICATA STREAM 3way handshake SYN resend different seq on SYN recv [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.4:36747 -> 192.168.5.3:80
11/17/2025-07:55:00.293548 [**] [1:2210004:3] SURICATA STREAM 3way handshake SYNACK resend with different ack [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.3:80 -> 192.168.5.4:36747
11/17/2025-07:55:00.294756 [**] [1:2210008:3] SURICATA STREAM 3way handshake SYN resend different seq on SYN recv [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.4:36811 -> 192.168.5.3:80
11/17/2025-07:55:00.294945 [**] [1:2210004:3] SURICATA STREAM 3way handshake SYNACK resend with different ack [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.3:80 -> 192.168.5.4:36811
11/17/2025-07:55:00.295625 [**] [1:2210008:3] SURICATA STREAM 3way handshake SYN resend different seq on SYN recv [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.4:36903 -> 192.168.5.3:80
11/17/2025-07:55:00.295682 [**] [1:2210004:3] SURICATA STREAM 3way handshake SYNACK resend with different ack [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.3:80 -> 192.168.5.4:36903
11/17/2025-07:55:00.296140 [**] [1:2210008:3] SURICATA STREAM 3way handshake SYN resend different seq on SYN recv [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.4:36849 -> 192.168.5.3:80
11/17/2025-07:55:00.296174 [**] [1:2210004:3] SURICATA STREAM 3way handshake SYNACK resend with different ack [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.3:80 -> 192.168.5.4:36849
11/17/2025-07:55:00.296524 [**] [1:2210008:3] SURICATA STREAM 3way handshake SYN resend different seq on SYN recv [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.4:36866 -> 192.168.5.3:80
11/17/2025-07:55:00.296525 [**] [1:2210008:3] SURICATA STREAM 3way handshake SYN resend different seq on SYN recv [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.4:36867 -> 192.168.5.3:80
11/17/2025-07:55:00.296728 [**] [1:2210008:3] SURICATA STREAM 3way handshake SYN resend different seq on SYN recv [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.4:36873 -> 192.168.5.3:80
11/17/2025-07:55:00.296813 [**] [1:2210004:3] SURICATA STREAM 3way handshake SYNACK resend with different ack [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.3:80 -> 192.168.5.4:36876
11/17/2025-07:55:00.296855 [**] [1:2210004:3] SURICATA STREAM 3way handshake SYNACK resend with different ack [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.5.3:80 -> 192.168.5.4:36877
^C
root@Ubantu:/var/log/suricata#

,

What Suricata Detected in this pictures?

The entries you see are alerts such as:

- ET SCAN Suspicious inbound to MySQL port 3306
- ET SCAN Suspicious inbound to MSSQL port 1433
- ET SCAN Suspicious inbound to Oracle SQL port 1521
- ET SCAN Potential VNC Scan 5800–5820
- ET SCAN Suspicious inbound to PostgreSQL port 5432

These alerts tell us:

- Suricata is successfully capturing packets
- Your Kali machine (192.168.5.4) is sending aggressive/suspicious traffic
- Victim machine (192.168.5.3) is logging those attacks
- Your IDS is functioning correctly

-These alerts indicate:

Port scanning

- High-volume SYN packets
- Abnormal traffic patterns

- This is exactly what happens during DoS attacks or reconnaissance before a DoS attack.

8. Evidence Collection

Capture-

- Screenshot of hping3 sending packets
- Screenshot of Suricata detecting DoS alert
- Logs from fast.log
- Extract from eve.json
- Suricata performance counter

10. Result

The project successfully demonstrated how Suricata detects a Denial-of-Service (DoS) attack in a controlled network environment. When the attacker machine (Kali Linux) generated a high volume of SYN flood packets using *hping3*, Suricata on the victim system accurately identified the abnormal traffic patterns. Multiple alerts were triggered in **fast.log** and **eve.json**, indicating suspicious inbound SYN packets coming from the attacker's IP address.

The logs showed clear evidence of increased traffic rates, repeated connection attempts, and rule-based detections such as **"ET DOS Possible SYN Flood Attack"**. Suricata's real-time monitoring confirmed its ability to recognize malicious behavior, classify the attack, and record all important details including timestamps, source/destination IPs, and signatures.

Overall, the results verify that Suricata is effective in detecting DoS attacks and provides timely alerts that can help administrators respond quickly and mitigate network threats.

,

11. Conclusion

This project successfully demonstrated the use of Suricata as an effective tool for detecting Denial-of-Service (DoS) attacks in a controlled network environment. By simulating a SYN flood attack from Kali Linux and monitoring the victim system with Suricata, the project showed how real-time traffic inspection and rule-based detection enable quick identification of abnormal network behavior. Suricata generated accurate alerts, captured detailed log entries, and provided clear evidence of the attack, highlighting its usefulness for intrusion detection and network security analysis. Overall, the project confirms that Suricata is a powerful and reliable IDS solution capable of enhancing network protection against DoS threats.